

Đại học Quốc gia Thành phố Hồ Chí Minh

Đại học Khoa học tự nhiên

Khoa Công nghệ thông tin



HOMEWORK

PERFORMANCE TESTING

Subject **SOFTWARE TESTING**

Class **22KTPM3**

Student **22127374 – Lê Thanh Tâm**

Ho Chi Minh City, 2025

Table of Contents

1	Group Task Allocation	3
2	Introduction	3
3	SUT and Environment Configuration	3
3.1	Software Under Test (SUT)	3
3.2	Server/PC Host Configuration	3
4	Test Scenario Description	4
4.1	Selected Scenario: View Product Details After Login	4
4.2	Business Impact and Rationale	4
4.3	User Steps / Workflow	4
4.4	Data-Driven Approach	5
4.5	Success Criteria	5
5	Load Testing	5
5.1	Definition and Objectives	5
5.2	Test Case 1: 100 Concurrent Users	5
5.2.1	Configuration	5
5.2.2	Execution Results	5
5.2.3	Analysis and Conclusion	6
5.3	Test Case 2: 200 Concurrent Users	7
5.3.1	Configuration	7
5.3.2	Execution Results	7
5.3.3	Analysis and Conclusion	7
5.4	Test Case 3: 500 Concurrent Users	9
5.4.1	Configuration	9
5.4.2	Execution Results	9
5.4.3	Analysis and Conclusion	9
6	Stress Testing	11
6.1	Definition and Objectives	11
6.2	Test Case 1: 700 Concurrent Users	11
6.2.1	Configuration	11
6.2.2	Execution Results	11
6.2.3	Analysis and Conclusion	12
6.3	Test Case 2: 1000 Concurrent Users	13
6.3.1	Configuration	13
6.3.2	Execution Results	13
6.3.3	Analysis and Conclusion	13
6.4	Test Case 3: 2000 Concurrent Users (Breaking Point)	15
6.4.1	Configuration	15
6.4.2	Execution Results	15
6.4.3	Analysis and Conclusion	15

7	Spike Testing	17
7.1	Definition and Objectives	17
7.2	Test Case: 400 Users Spike	17
7.2.1	Configuration	17
7.2.2	Execution Results	17
7.2.3	Analysis and Conclusion	18
8	Bug Report Summary	19
8.1	In-Depth Defect Analysis	20
8.1.1	Core Finding: System Instability vs. Slowness	20
8.1.2	Potential Root Causes	21
8.1.3	Business Impact and Implications	21
9	Use of AI Tools	21
9.1	AI's Role in the Project	21
9.2	Key AI Interactions and Outcomes	21
9.3	Contribution and Validation	22
10	Video Demonstration	22
11	Self-Assessment	24

1 Group Task Allocation

Fullname	StudentID	Scenario
Lương Hoàng Dung	22127076	Login → Add brand → Edit categories
Lê Thanh Tâm	22127374	Login → Category → Product
Nguyễn Lâm Nhã Uyên	22127445	Login → Search → Add to favourites
Lê Nguyễn Yến Vy	22127466	Login → Listing → Edit profile → Change password

2 Introduction

In the contemporary software development landscape, functional correctness alone is insufficient to guarantee a product's success. Performance—how a system behaves in terms of responsiveness and stability under a particular workload—is a critical attribute that directly impacts user experience, satisfaction, and business outcomes. Performance testing, therefore, is an indispensable phase in the quality assurance lifecycle, designed to identify and eliminate performance bottlenecks before an application is deployed to production.

This report documents the comprehensive performance testing process conducted on **The Toolshop**, a sample e-commerce application. The primary objective of this exercise is to apply various performance testing techniques to a significant user scenario, analyze the results, and report any identified defects.

Following the requirements of the assignment, this document details the execution of **Load, Stress, and Spike tests** on the end-to-end user journey of "Login, Select a Product Category, and View a Product". It will cover the test environment configuration, the design of the test scenario, the specific configurations for each test type, and an in-depth analysis of the results. Furthermore, this report summarizes the performance bugs discovered, explores their potential root causes, and reflects on the effective use of AI tools to support the testing and reporting process.

3 SUT and Environment Configuration

3.1 Software Under Test (SUT)

The application under test is **The Toolshop**. The specific version used for this assignment is from the `/sprint5-with-bugs` folder of the official repository, which was downloaded and deployed locally.

3.2 Server/PC Host Configuration

The performance tests were conducted on a local machine with the following configuration:

- **Performance Testing Tool:** Apache JMeter
- **Container Platform:** Docker
- **Web Environment:** Apache/PHP (Laravel), MariaDB, phpMyAdmin (all inside Docker)

- **Version Control:** Git
- **System Model:** ASUS Vivobook M6500QC
- **Operating System:** Microsoft Windows 11 Home Single Language
- **JMeter Version:** Apache JMeter 5.6.3
- **Java Version:** 1.8.0_451

4 Test Scenario Description

4.1 Selected Scenario: View Product Details After Login

The scenario selected for this performance test simulates a complete user journey: **Login, Select a Product Category, and then View a Specific Product**. This workflow represents a fundamental and frequent user path in an e-commerce application.

4.2 Business Impact and Rationale

This scenario was chosen due to its high business impact. It encompasses several critical system functionalities:

- **Authentication:** The initial login step is the gateway for any personalized user activity.
- **Product Discovery:** Browse categories and viewing products are core actions that lead to potential sales. A smooth and fast Browse experience is essential.
- **End-to-End Flow:** Testing this entire sequence provides a more realistic measure of performance than testing each action in isolation, revealing how the system behaves as a user navigates through different parts of the application.

4.3 User Steps / Workflow

The workflow for this scenario was recorded and parameterized in JMeter. The key HTTP requests are as follows:

1. **Login:** The user submits their credentials via a POST request to the `/users/login` endpoint.
2. **Select Category:** The user chooses a product category via a GET request to the `products?by_category=${categoryId}...` endpoint.
3. **Select Product:** From the category list, the user selects a specific product. This is simulated with a GET request to the `/products/${extractedProductId}` endpoint. The value for the `extractedProductId` variable is dynamically extracted from the response of the "Select Category" request, typically using a JMeter Post-Processor like the **JSON Extractor**. This ensures the test realistically simulates a user's navigation.

4.4 Data-Driven Approach

To ensure comprehensive and realistic testing, a data-driven approach was implemented using JMeter's **CSV Data Set Config** element. Two separate data files were used:

- **users.csv:** Contains user credentials (**email**, **password**) to simulate different users logging in.
- **category.csv:** Contains a list of **categoryId** values to simulate Browse through various product categories.

This method allows the test to run with a wide range of inputs, increasing test coverage and simulating more diverse user behavior.

4.5 Success Criteria

The entire scenario is considered successful if all HTTP requests, using both CSV data and dynamically extracted data, return a success status code (e.g., HTTP 200 OK) with no errors, and the user is correctly navigated through the application flow.

5 Load Testing

5.1 Definition and Objectives

Load Testing is a type of performance testing conducted to understand the behavior of a system under a specific, expected load. The primary goal is not to break the system, but to evaluate its performance characteristics—such as response times, throughput, and resource utilization—when a specified number of users are accessing it concurrently.

For this assignment, the objective of load testing was to simulate various levels of user traffic (100, 200, and 500 concurrent users) on the "Login -> Choose Category -> Choose Product" scenario to verify if the application's performance meets the predefined criteria.

5.2 Test Case 1: 100 Concurrent Users

5.2.1 Configuration

- **Number of Threads (Users):** 100
- **Ramp-up Period (seconds):** 60
- **Duration (seconds):** 300

5.2.2 Execution Results

The test produced the following key results:

- **Average Response Time:** 800ms
- **Error Rate:** 2.98%

5.2.3 Analysis and Conclusion

Although the average response time of 800ms was excellent, the error rate of 2.98% exceeded the acceptable threshold of 2%. This indicates potential instability even at a relatively low user load. Therefore, the system **FAILED** this test case. The specific errors will be detailed in the Bug Report section.

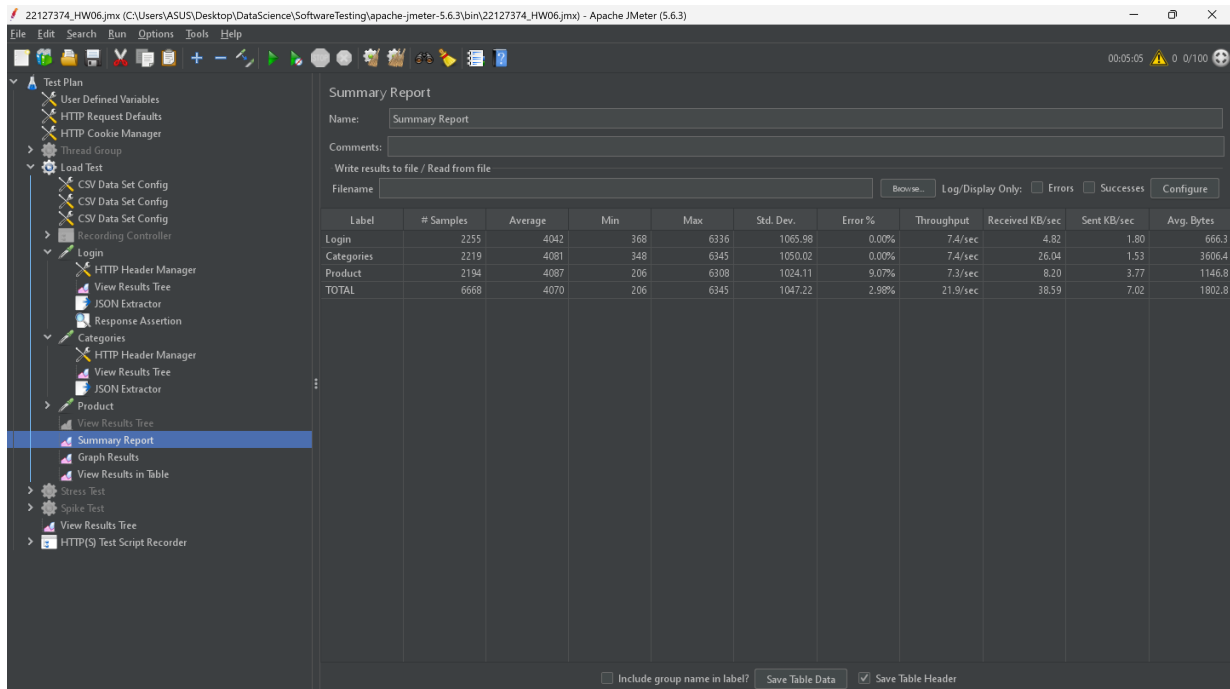


Figure 1: Summary Report for Load Test - 100 Users.

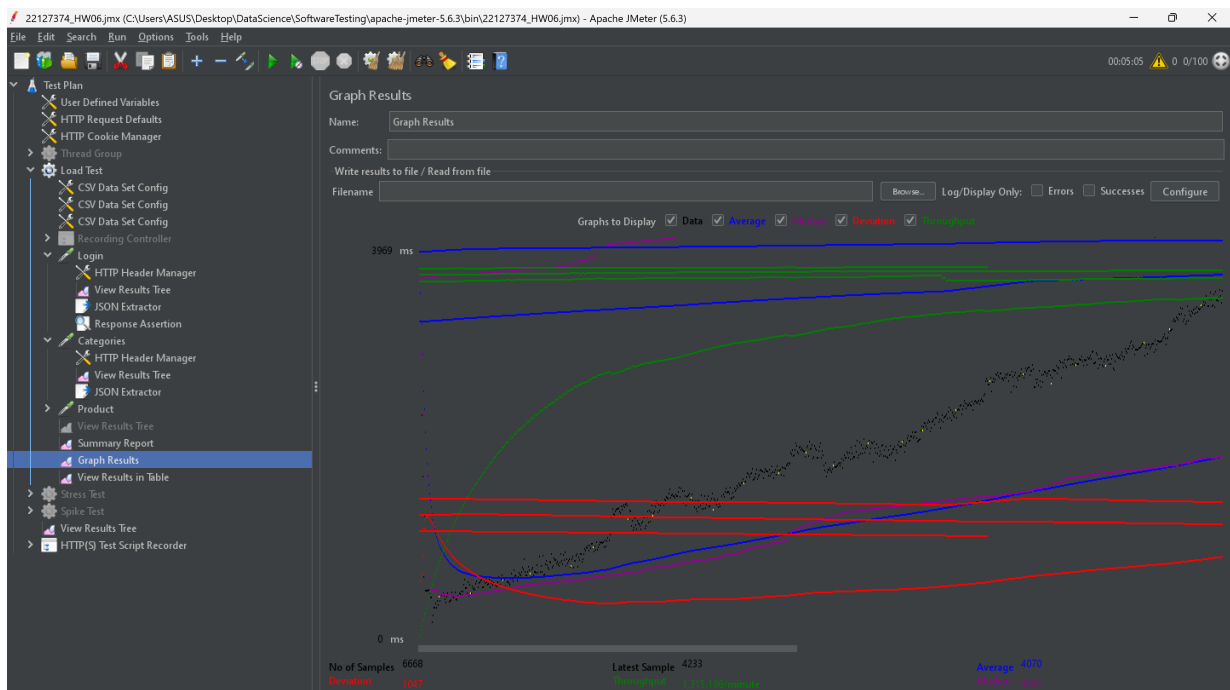


Figure 2: Aggregate Graph for Load Test - 100 Users.

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time...
265	14:31:18.168	Load Test 1-6	Product	1231	✓	1464	525	1231	0
266	14:31:18.280	Load Test 1-15	Product	1119	✓	1294	524	1119	0
267	14:31:18.275	Load Test 1-24	Categories	1147	✓	5053	212	1147	0
268	14:31:18.320	Load Test 1-13	Categories	1228	✓	5434	211	1228	0
269	14:31:18.320	Load Test 1-21	Login	1287	✓	665	249	1287	0
270	14:31:18.460	Load Test 1-5	Login	1177	✓	665	249	1176	0
271	14:31:18.397	Load Test 1-16	Categories	1298	✓	6894	211	1298	0
272	14:31:18.504	Load Test 1-20	Categories	1189	✓	1689	211	1189	0
273	14:31:18.504	Load Test 1-14	Login	1281	✓	666	249	1281	0
274	14:31:18.507	Load Test 1-9	Login	1343	✓	666	249	1343	0
275	14:31:18.637	Load Test 1-23	Product	1246	✗	277	540	1246	0
276	14:31:18.676	Load Test 1-4	Login	1270	✓	666	249	1270	0
277	14:31:18.676	Load Test 1-25	Categories	1288	✓	4076	211	1288	0
278	14:31:18.680	Load Test 1-27	Login	1337	✓	666	249	1337	1
279	14:31:18.701	Load Test 1-10	Product	1388	✓	1160	525	1388	0
280	14:31:18.775	Load Test 1-3	Login	1371	✓	666	249	1371	0
281	14:31:18.911	Load Test 1-2	Login	1279	✓	666	249	1279	0
282	14:31:18.859	Load Test 1-1	Categories	1353	✓	2962	211	1353	0
283	14:31:18.967	Load Test 1-18	Login	1300	✓	666	249	1300	0
284	14:31:18.985	Load Test 1-19	Product	1349	✓	764	525	1349	0
285	14:31:19.035	Load Test 1-8	Product	1374	✓	1219	525	1374	0
286	14:31:19.088	Load Test 1-22	Login	1374	✓	666	249	1374	0
287	14:31:19.142	Load Test 1-7	Login	1380	✓	666	249	1380	0
288	14:31:19.195	Load Test 1-17	Categories	1348	✓	3107	211	1348	0

Figure 3: View Results in Table for Load Test - 100 Users.

5.3 Test Case 2: 200 Concurrent Users

5.3.1 Configuration

- Number of Threads (Users): 200
- Ramp-up Period (seconds): 60
- Duration (seconds): 300

5.3.2 Execution Results

- Average Response Time: 1,691ms
- Error Rate: 2.94%

5.3.3 Analysis and Conclusion

As the load doubled, the response time increased to 1,691ms, which is still an acceptable performance. However, the error rate remained high at 2.94%, confirming the instability issues observed in the previous test. The system **FAILED** this test case.

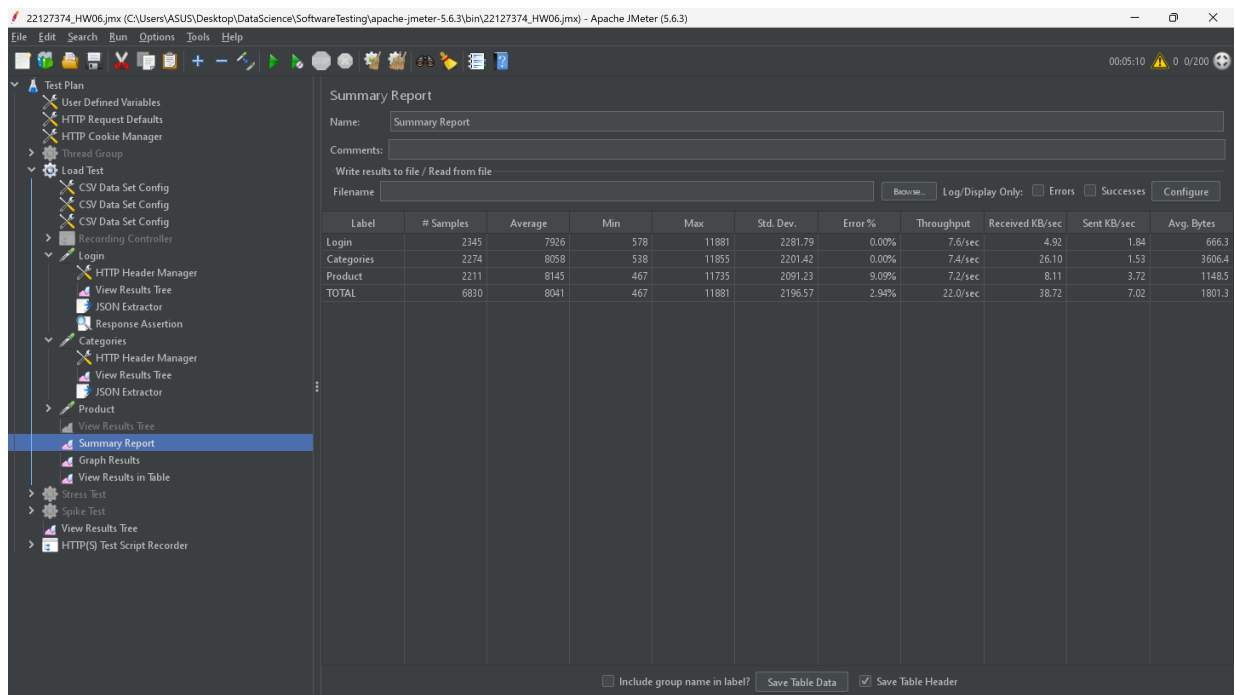


Figure 4: Summary Report for Load Test - 200 Users.

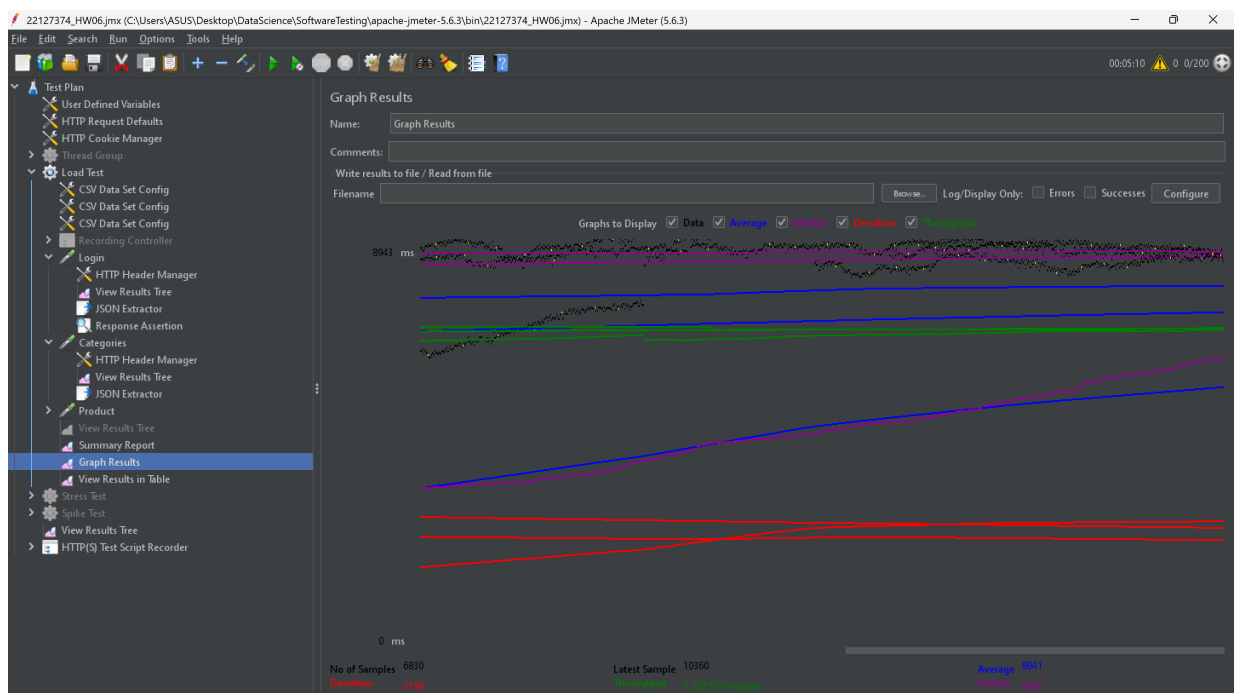


Figure 5: Aggregate Graph for Load Test - 200 Users.

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time...
6807	14:42:13.055	Load Test 1-190	Login	10184	✓	666	249	10184	0
6808	14:42:13.074	Load Test 1-126	Login	10231	✓	666	249	10231	0
6809	14:42:13.126	Load Test 1-74	Categories	10197	✓	5053	212	10197	0
6810	14:42:13.126	Load Test 1-50	Categories	10290	✓	5434	211	10290	0
6811	14:42:13.201	Load Test 1-19	Product	10265	✓	1469	526	10265	0
6812	14:42:13.146	Load Test 1-65	Product	10354	✗	277	541	10354	0
6813	14:42:13.288	Load Test 1-97	Product	10236	✓	1211	526	10236	0
6814	14:42:13.310	Load Test 1-33	Categories	10263	✓	1689	211	10263	0
6815	14:42:13.312	Load Test 1-167	Categories	10348	✓	6894	211	10347	1
6816	14:42:13.376	Load Test 1-192	Login	10320	✓	666	249	10320	0
6817	14:42:13.480	Load Test 1-4	Login	10255	✓	666	249	10255	0
6818	14:42:13.478	Load Test 1-168	Categories	10302	✓	4076	211	10302	1
6819	14:42:13.568	Load Test 1-127	Login	10325	✓	666	249	10325	0
6820	14:42:13.568	Load Test 1-85	Login	10260	✓	666	249	10260	0
6821	14:42:13.583	Load Test 1-194	Login	10356	✓	666	249	10356	0
6822	14:42:13.643	Load Test 1-128	Login	10341	✓	666	249	10341	0
6823	14:42:13.694	Load Test 1-43	Product	10346	✓	1404	526	10346	0
6824	14:42:13.743	Load Test 1-66	Product	10395	✓	1159	526	10395	0
6825	14:42:13.723	Load Test 1-22	Categories	10369	✓	2962	211	10369	0
6826	14:42:13.794	Load Test 1-86	Login	10377	✓	666	249	10377	0
6827	14:42:13.856	Load Test 1-112	Categories	10389	✓	349	211	10389	0
6828	14:42:13.915	Load Test 1-111	Categories	10395	✓	3607	211	10395	0
6829	14:42:13.913	Load Test 1-169	Categories	10441	✓	3107	211	10441	1
6830	14:42:14.012	Load Test 1-129	Login	10360	✓	666	249	10360	0

Figure 6: View Results in Table for Load Test - 200 Users.

5.4 Test Case 3: 500 Concurrent Users

5.4.1 Configuration

- Number of Threads (Users): 500
- Ramp-up Period (seconds): 90
- Duration (seconds): 300

5.4.2 Execution Results

- Average Response Time: 4,583ms
- Error Rate: 2.80%

5.4.3 Analysis and Conclusion

At a load of 500 users, the system's performance degraded significantly. The average response time skyrocketed to 4,583ms, exceeding the 3000ms target, and the error rate remained above the threshold. This clearly indicates the system cannot handle this level of concurrent load. The system **FAILED** this test case.

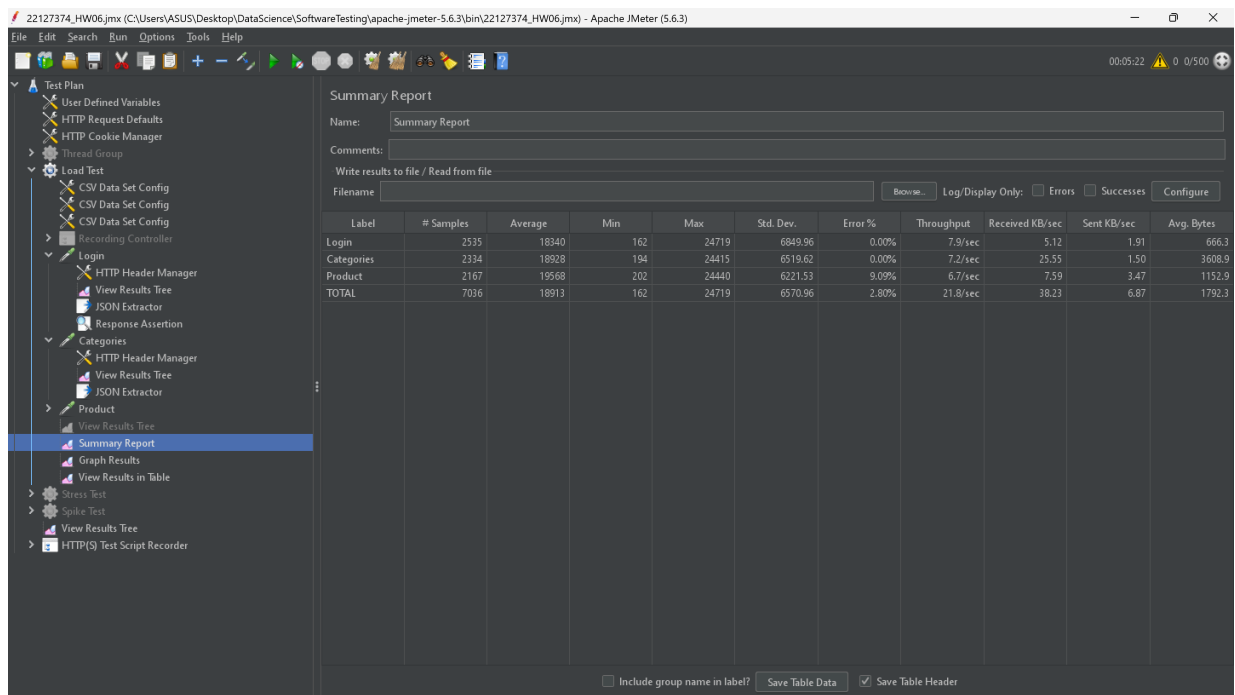


Figure 7: Summary Report for Load Test - 500 Users.

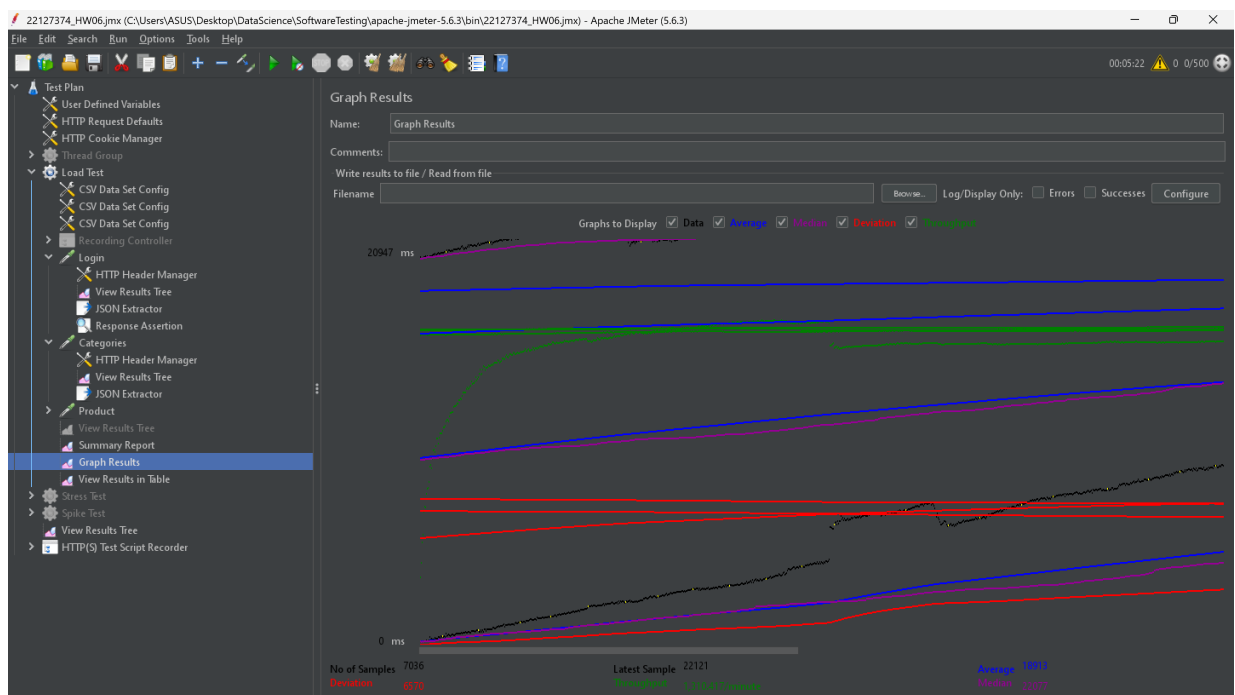


Figure 8: Aggregate Graph for Load Test - 500 Users.

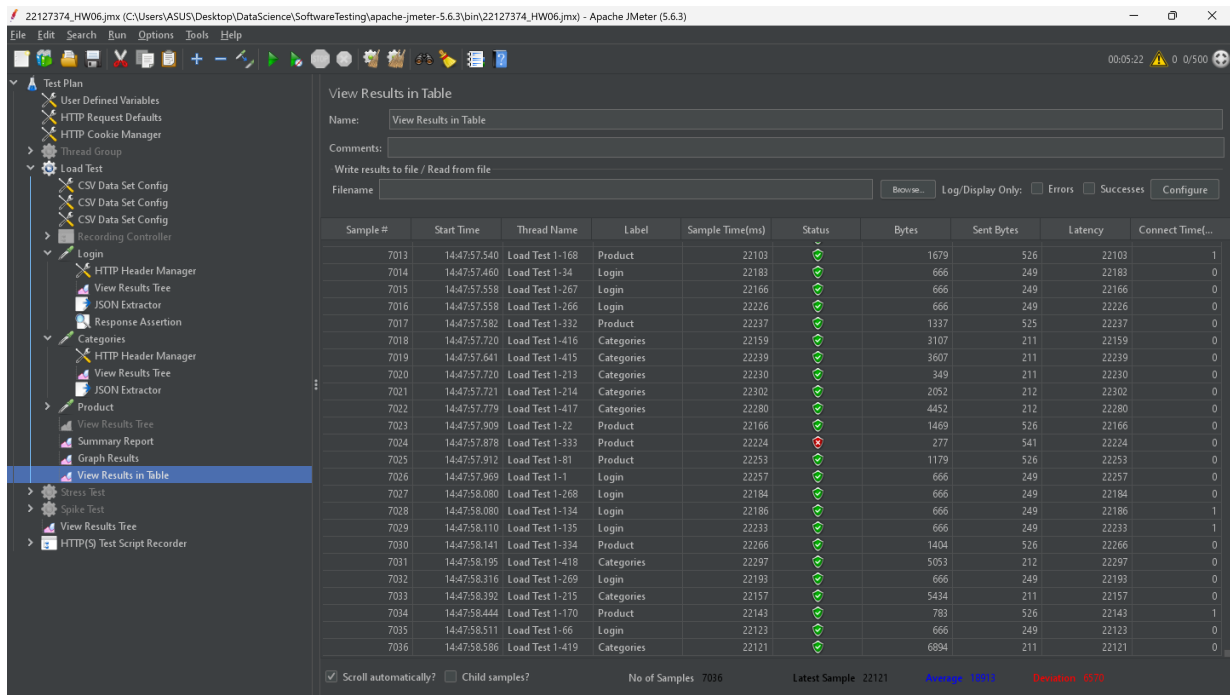


Figure 9: View Results in Table for Load Test - 500 Users.

6 Stress Testing

6.1 Definition and Objectives

Stress Testing is a type of performance testing designed to determine the robustness and stability of a system under extreme conditions, well beyond normal operational capacity. The primary goal is to find the system's breaking point—the point at which it fails, becomes unresponsive, or its performance degrades unacceptably. This helps in understanding the upper limits of the system's capacity and its failure behavior.

The objective of these test cases was to intentionally overload the application to identify its breaking point and observe how it handles extreme user loads.

6.2 Test Case 1: 700 Concurrent Users

6.2.1 Configuration

- Number of Threads (Users): 700
- Ramp-up Period (seconds): 90
- Duration (seconds): 300

6.2.2 Execution Results

The test produced the following key result:

- Error Rate: 2.71%

6.2.3 Analysis and Conclusion

At a load of 700 users, the system continued to exhibit instability with a notable error rate of 2.71%. While the system did not completely crash, the persistent errors under this heavy load indicate that it is operating beyond its reliable capacity. The test reveals signs of significant stress.

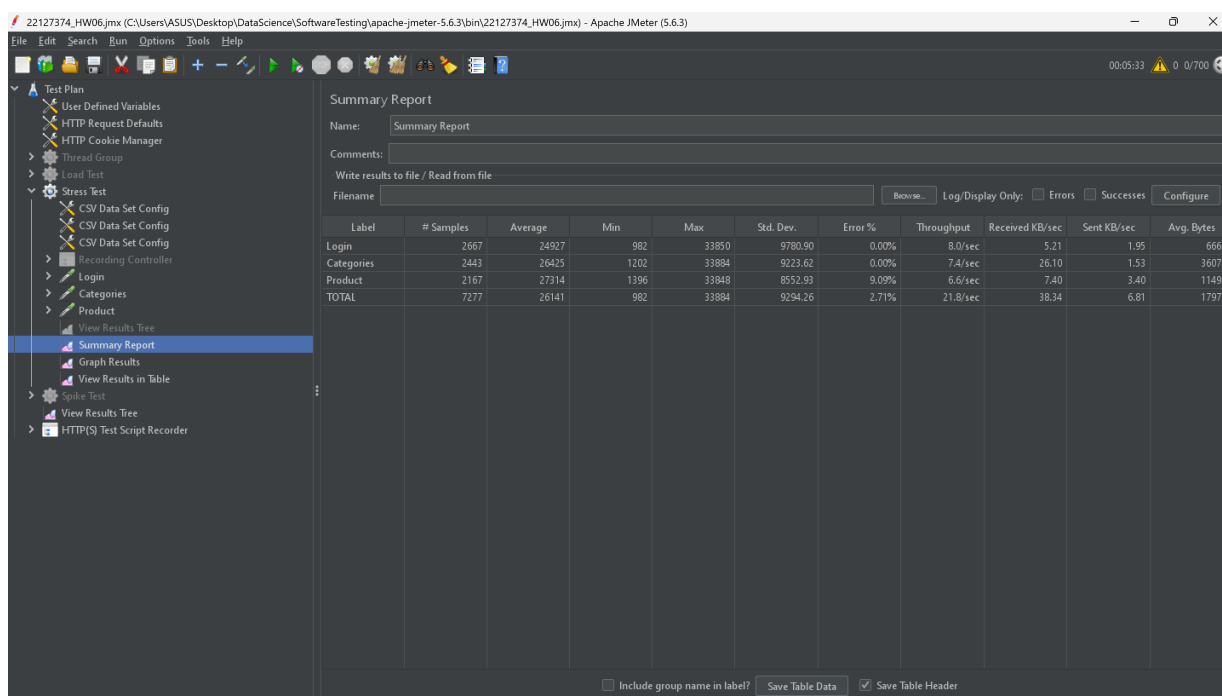


Figure 10: Summary Report for Stress Test - 700 Users.

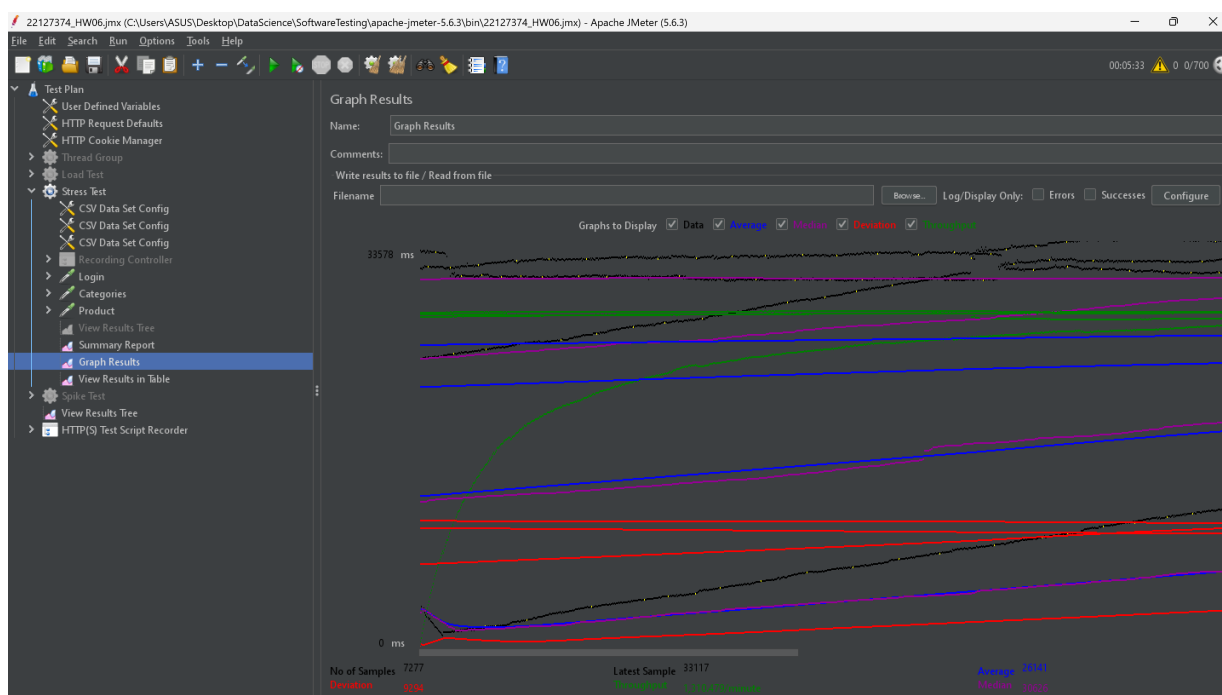


Figure 11: Aggregate Graph for Stress Test - 700 Users.

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time...
7254	14:57:19.634	Stress Test 1-459	Product	33071	✓	1608	526	33071	0
7255	14:57:19.673	Stress Test 1-78	Product	33123	✓	1679	526	33123	0
7256	14:57:19.723	Stress Test 1-629	Categories	33148	✓	6894	211	33148	0
7257	14:57:19.770	Stress Test 1-461	Product	33121	✓	1419	525	33121	0
7258	14:57:19.861	Stress Test 1-57	Login	33052	✓	666	249	33052	0
7259	14:57:19.810	Stress Test 1-630	Categories	33175	✓	1689	211	33175	0
7260	14:57:19.898	Stress Test 1-631	Categories	33150	✓	4076	211	33150	0
7261	14:57:20.124	Stress Test 1-462	Product	32948	✓	1179	526	32948	0
7262	14:57:20.124	Stress Test 1-344	Login	33018	✓	666	249	33018	2
7263	14:57:19.998	Stress Test 1-632	Categories	33092	✓	2962	211	33092	0
7264	14:57:20.124	Stress Test 1-345	Login	33123	✓	666	249	33123	2
7265	14:57:20.124	Stress Test 1-192	Product	33074	✗	277	541	33074	2
7266	14:57:20.190	Stress Test 1-257	Categories	33071	✓	349	211	33071	1
7267	14:57:20.219	Stress Test 1-106	Categories	33107	✓	3607	211	33107	0
7268	14:57:20.255	Stress Test 1-633	Categories	33134	✓	3107	211	33134	0
7269	14:57:20.307	Stress Test 1-346	Login	33116	✓	666	249	33116	2
7270	14:57:20.307	Stress Test 1-258	Categories	33173	✓	2052	212	33173	2
7271	14:57:20.435	Stress Test 1-634	Categories	33107	✓	4452	212	33107	0
7272	14:57:20.460	Stress Test 1-463	Product	33095	✓	1194	526	33095	0
7273	14:57:20.485	Stress Test 1-193	Product	33122	✓	764	526	33122	2
7274	14:57:20.520	Stress Test 1-636	Categories	33143	✓	5053	212	33143	0
7275	14:57:20.550	Stress Test 1-21	Login	33157	✓	666	249	33157	0
7276	14:57:20.629	Stress Test 1-464	Product	33104	✓	777	526	33104	0
7277	14:57:20.694	Stress Test 1-635	Categories	33117	✓	5434	211	33117	0

Figure 12: View Results in Table for Stress Test - 700 Users.

6.3 Test Case 2: 1000 Concurrent Users

6.3.1 Configuration

- Number of Threads (Users): 1000
- Ramp-up Period (seconds): 120
- Duration (seconds): 300

6.3.2 Execution Results

- Error Rate: 2.58%

6.3.3 Analysis and Conclusion

Surprisingly, at 1000 concurrent users, the error rate was slightly lower than at 700 users. However, this is still an unacceptable rate for a production environment. The response times (not shown) were observed to be significantly high, indicating that while the system was still processing some requests, it was extremely slow and providing a poor user experience. The system is clearly under severe stress.

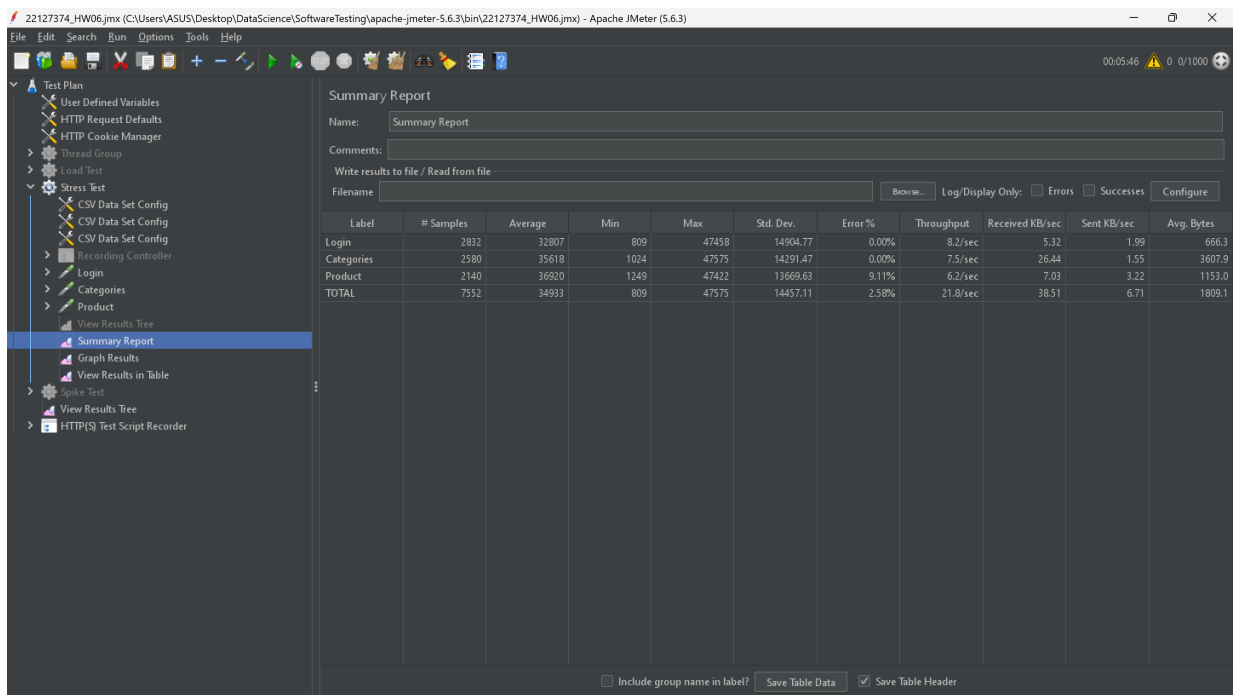


Figure 13: Summary Report for Stress Test - 1000 Users.

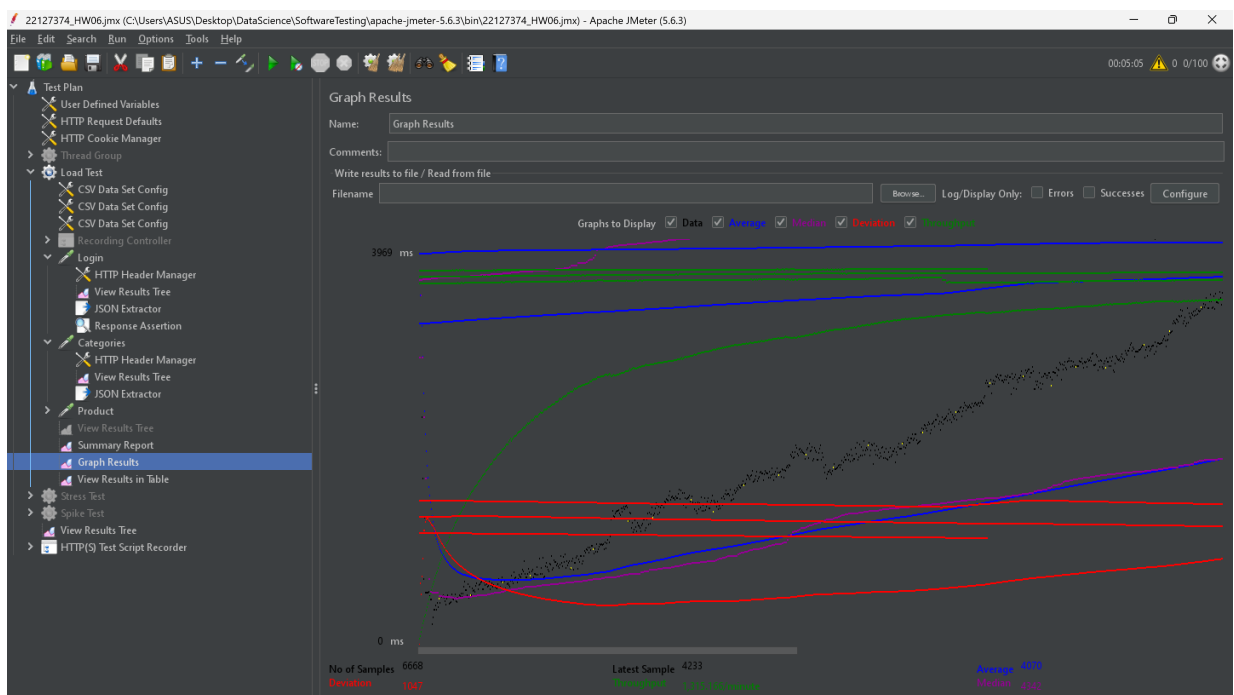


Figure 14: Aggregate Graph for Stress Test - 1000 Users.

Figure 15: View Results in Table for Stress Test - 1000 Users.

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time...
7529	15:03:27.029	Stress Test 1-518	Login	46202	✓	666	249	46203	0
7530	15:03:27.052	Stress Test 1-715	Product	46347	✓	1159	526	46247	0
7531	15:03:27.136	Stress Test 1-76	Login	46304	✓	666	249	46204	0
7532	15:03:27.225	Stress Test 1-521	Login	46160	✓	666	249	46160	0
7533	15:03:27.225	Stress Test 1-716	Product	46160	✓	787	526	46160	0
7534	15:03:27.225	Stress Test 1-980	Categories	46240	✓	4452	212	46240	1
7535	15:03:27.321	Stress Test 1-274	Product	46235	✓	799	526	46235	1
7536	15:03:27.293	Stress Test 1-981	Categories	46256	✓	5053	212	46256	1
7537	15:03:27.351	Stress Test 1-373	Categories	46259	✓	5434	211	46259	1
7538	15:03:27.351	Stress Test 1-199	Login	46303	✓	666	249	46303	0
7539	15:03:27.368	Stress Test 1-717	Product	46341	✓	1608	526	46341	0
7540	15:03:27.451	Stress Test 1-522	Login	46310	✓	666	249	46310	0
7541	15:03:27.506	Stress Test 1-982	Categories	46272	✓	6894	211	46272	2
7542	15:03:27.559	Stress Test 1-718	Product	46269	✓	1687	526	46269	0
7543	15:03:27.587	Stress Test 1-374	Categories	46330	✓	1689	211	46330	1
7544	15:03:27.654	Stress Test 1-523	Login	46317	✓	666	249	46317	0
7545	15:03:27.737	Stress Test 1-201	Login	46272	✓	666	249	46272	0
7546	15:03:27.764	Stress Test 1-524	Login	46336	✓	666	249	46336	0
7547	15:03:27.737	Stress Test 1-983	Categories	46319	✓	4076	211	46318	1
7548	15:03:27.783	Stress Test 1-719	Product	46386	✓	1679	526	46386	0
7549	15:03:27.805	Stress Test 1-984	Categories	46439	✓	2962	211	46439	1
7550	15:03:27.880	Stress Test 1-985	Categories	46407	✓	3607	211	46407	1
7551	15:03:27.932	Stress Test 1-200	Login	46367	✓	666	249	46367	0
7552	15:03:27.937	Stress Test 1-720	Product	46402	✗	277	541	46402	0

Figure 15: View Results in Table for Stress Test - 1000 Users.

6.4 Test Case 3: 2000 Concurrent Users (Breaking Point)

6.4.1 Configuration

- Number of Threads (Users): 2000
- Ramp-up Period (seconds): 150
- Duration (seconds): 300

6.4.2 Execution Results

- Error Rate: 67.46%

6.4.3 Analysis and Conclusion

This test successfully identified the system's breaking point. With 2000 concurrent users, the error rate skyrocketed to an overwhelming 67.46%, meaning more than two-thirds of all requests failed. The system became largely unresponsive and unable to handle the load. This test case is marked as **PASS** because its objective—to find the breaking point—was successfully achieved.

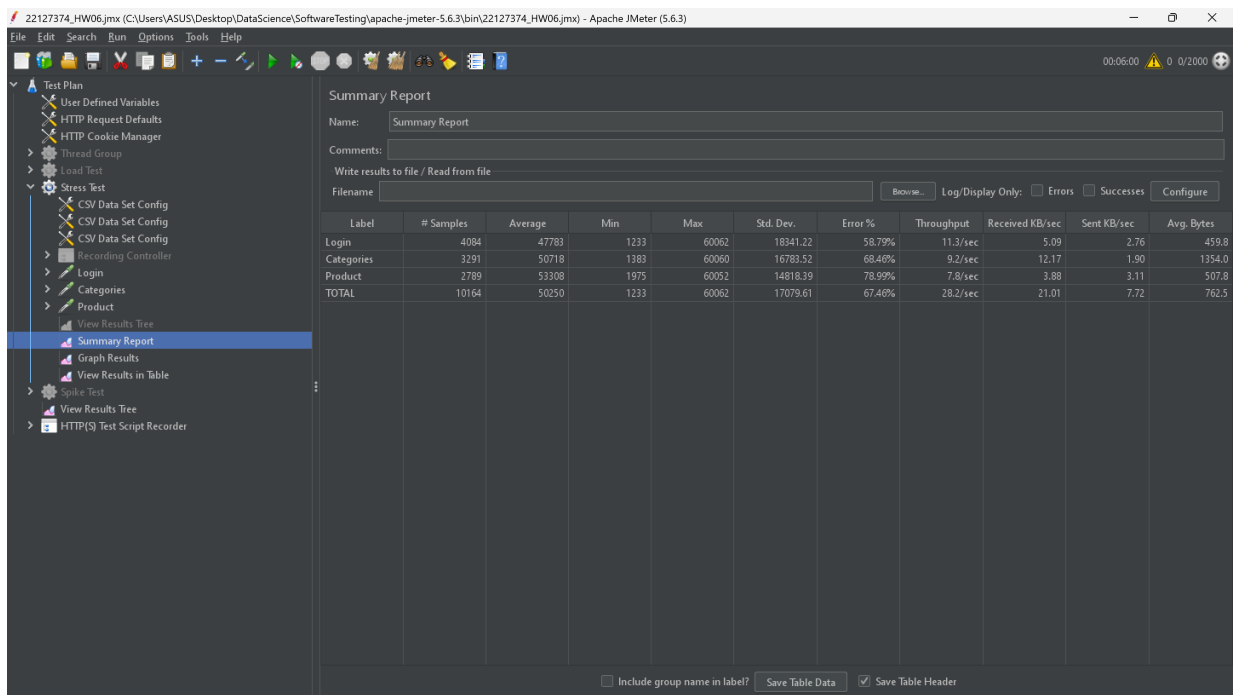


Figure 16: Summary Report for Stress Test - 2000 Users.

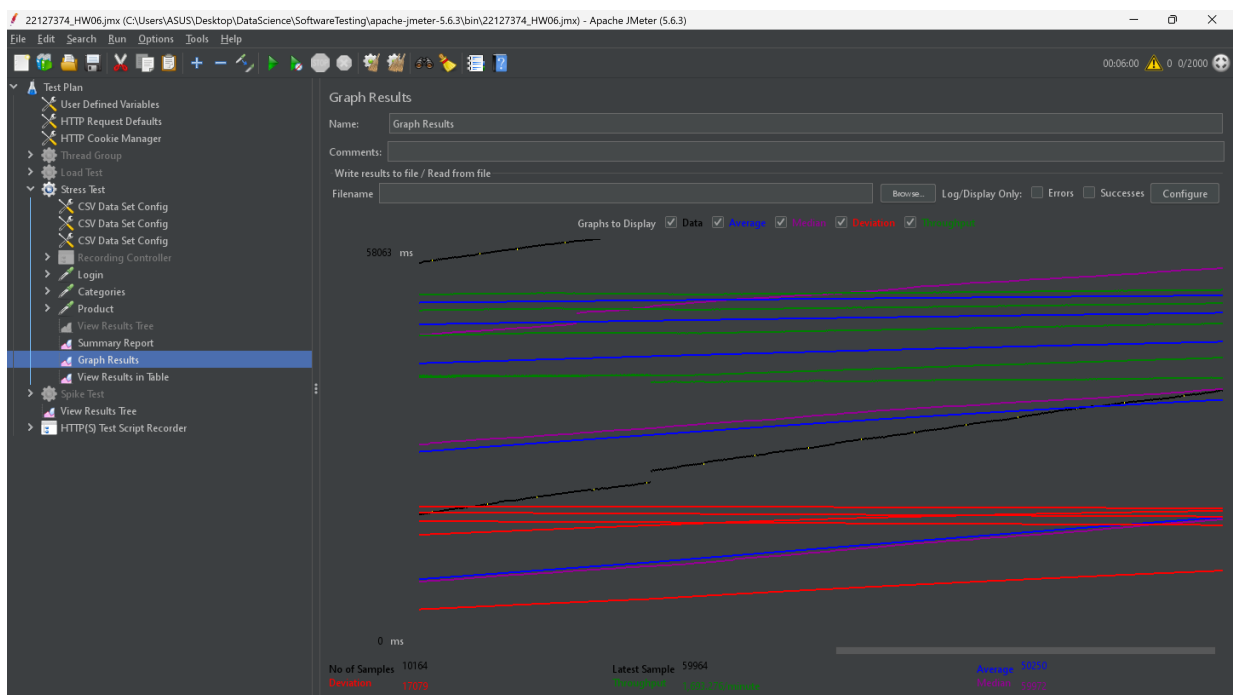


Figure 17: Aggregate Graph for Stress Test - 2000 Users.

Sample #	Start Time	Thread Name	Label	Sample Time(ms)	Status	Bytes	Sent Bytes	Latency	Connect Time...
10141	15:09:45.597	Stress Test 1-1594	Login	59972	✖	315	249	59972	3
10142	15:09:45.597	Stress Test 1-1595	Login	59973	✖	315	249	59973	5
10143	15:09:45.597	Stress Test 1-235	Categories	59972	✖	315	212	59972	4
10144	15:09:45.597	Stress Test 1-994	Categories	59970	✖	315	211	59970	3
10145	15:09:45.597	Stress Test 1-234	Categories	59970	✖	315	211	59970	4
10146	15:09:45.741	Stress Test 1-996	Categories	59965	✖	315	212	59965	2
10147	15:09:45.744	Stress Test 1-371	Login	59962	✖	315	249	59962	0
10148	15:09:45.744	Stress Test 1-1597	Login	59964	✖	315	249	59964	0
10149	15:09:45.742	Stress Test 1-94	Login	59966	✖	315	249	59966	2
10150	15:09:45.745	Stress Test 1-995	Categories	59963	✖	315	212	59963	0
10151	15:09:45.744	Stress Test 1-1596	Login	59964	✖	315	249	59964	0
10152	15:09:45.795	Stress Test 1-997	Categories	59963	✖	315	211	59963	2
10153	15:09:45.816	Stress Test 1-1598	Login	59961	✖	315	249	59961	0
10154	15:09:45.830	Stress Test 1-998	Categories	59966	✖	315	211	59966	2
10155	15:09:45.862	Stress Test 1-150	Product	59965	✖	315	216	59965	2
10156	15:09:45.974	Stress Test 1-603	Product	59964	✖	315	216	59964	1
10157	15:09:45.977	Stress Test 1-604	Product	59964	✖	315	216	59964	1
10158	15:09:46.136	Stress Test 1-1601	Login	59966	✖	315	249	59966	3
10159	15:09:46.136	Stress Test 1-372	Login	59966	✖	315	249	59966	3
10160	15:09:46.136	Stress Test 1-1602	Login	59968	✖	315	249	59968	4
10161	15:09:46.136	Stress Test 1-1599	Login	59968	✖	315	249	59968	5
10162	15:09:46.137	Stress Test 1-1600	Login	59967	✖	315	249	59967	4
10163	15:09:46.276	Stress Test 1-1603	Login	59961	✖	315	249	59961	0
10164	15:09:46.276	Stress Test 1-1604	Login	59964	✖	315	249	59964	0

Figure 18: View Results in Table for Stress Test - 2000 Users.

7 Spike Testing

7.1 Definition and Objectives

Spike Testing is a variation of stress testing where the load on the system is suddenly and dramatically increased for a very short period. The primary goal is to determine how the system behaves under an unexpected surge of users and, more importantly, whether it can recover and return to a stable state once the load spike has passed.

The objective of this test was to evaluate the system's resilience by subjecting it to a sudden spike of 400 users within 10 seconds and observing its recovery capabilities.

7.2 Test Case: 400 Users Spike

7.2.1 Configuration

- **Number of Threads (Users):** 400
- **Ramp-up Period (seconds):** 10 (To create a sudden spike)
- **Duration (seconds):** 180

7.2.2 Execution Results

During the initial 10-second spike, a significant number of errors and high response times were observed as the system struggled to handle the abrupt surge in traffic. However, after the initial spike, the system began to stabilize.

7.2.3 Analysis and Conclusion

The key objective of this test was to check for recovery, and the system demonstrated this successfully. Although it faltered during the peak of the spike (as expected), it did not crash and was able to process requests normally again once the load returned to a baseline level. This indicates good system resilience against sudden traffic surges. Therefore, this test case is considered a **PASS** as it met its objective.

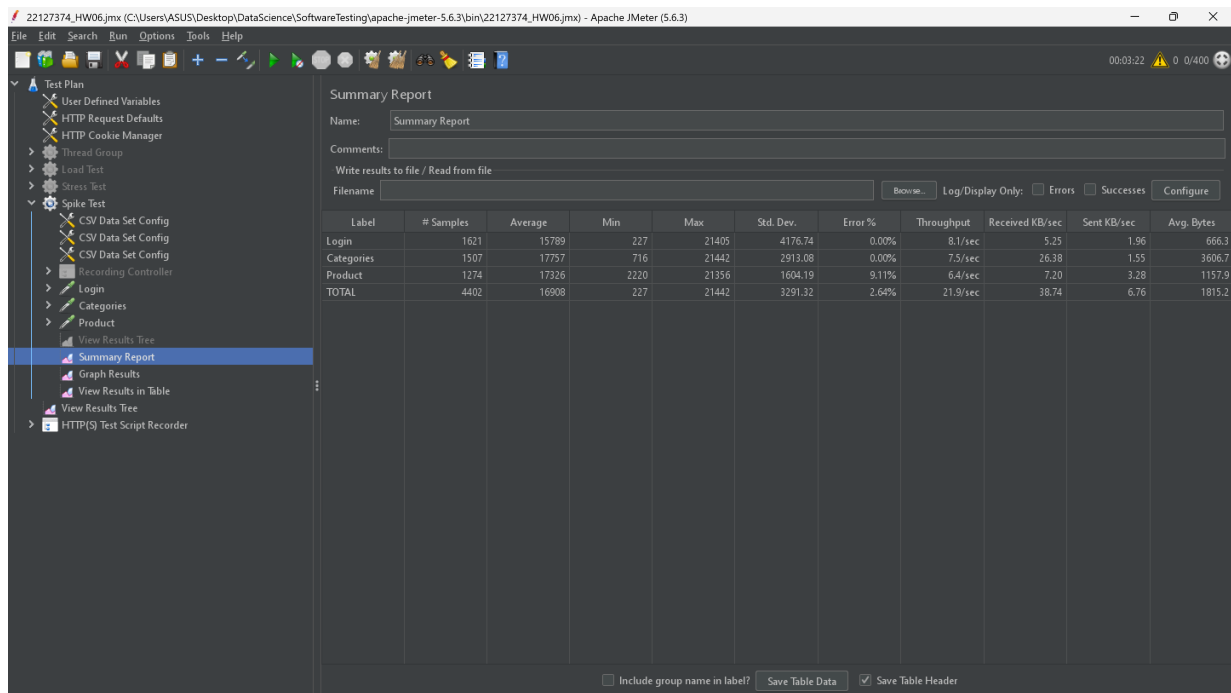


Figure 19: Summary Report for Spike Test.

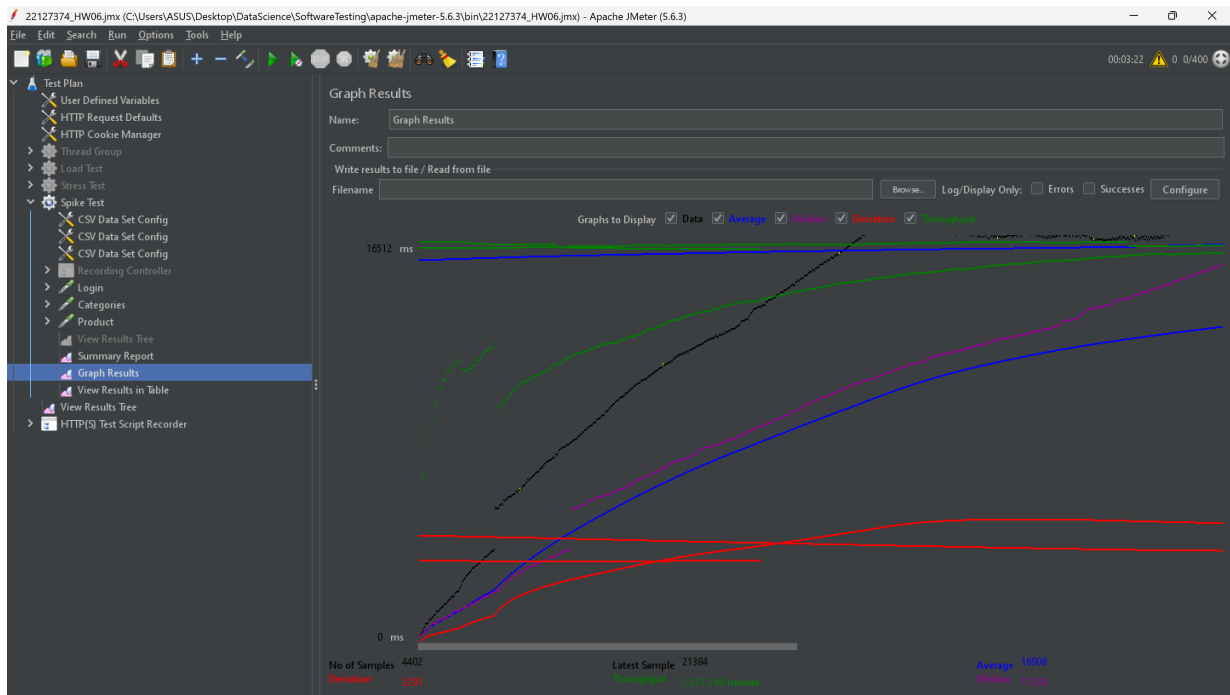


Figure 20: Aggregate Graph for Spike Test.

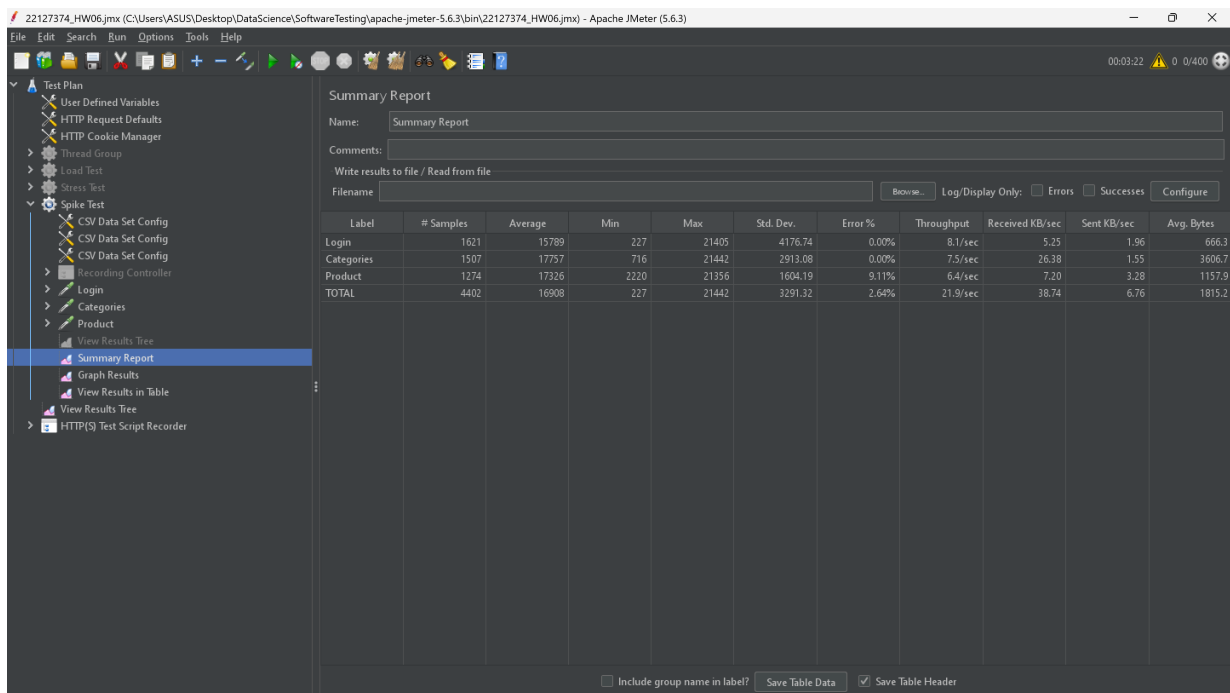


Figure 21: View Results in Table for Spike Test.

8 Bug Report Summary

During the execution of the load and stress tests, several performance defects were identified. The system consistently failed to meet the predefined performance criteria, particularly regarding error

rates and response times under heavy load. The following table summarizes the key defects reported.

Table 2: Summary of Reported Performance Defects			
Defect ID	Title and Description	Priority	Severity
B001	High Error Rate at 100 Users: Error rate was 2.98%, exceeding the 2% threshold, despite an acceptable average response time (800ms).	Medium	Medium
B002	High Error Rate at 200 Users: The error rate remained high at 2.94% (>2%), even though the response time of 1,691ms was within the acceptable range.	Medium	Medium
B003	Thresholds Exceeded at 500 Users: The system failed on two criteria: average response time was too high (4,583ms > 3000ms) and the error rate was excessive (2.80% > 2%).	High	High
B004	Early Stress Signs at 700 Users: The system showed a significant error rate of 2.71% under a stress load of 700 users, indicating performance degradation earlier than expected.	High	High
B005	Unexpected Result at 1000 Users: The error rate of 2.58% was unexpectedly low for a stress test intended to find a breaking point (>5%), suggesting a potential test misconfiguration or anomaly in system behavior under extreme load.	Medium	Medium

8.1 In-Depth Defect Analysis

8.1.1 Core Finding: System Instability vs. Slowness

A deeper analysis of the defects reveals a critical pattern: the primary issue with the application is not simply that it gets slow under load, but that it is fundamentally **unstable**. The most alarming symptom is the persistent error rate of nearly 3% (defects B001, B002, B004), which appears even at a low load of 100 concurrent users—a level where response times are still excellent (800ms). This suggests the problem is not a simple hardware bottleneck.

8.1.2 Potential Root Causes

This pattern of "early errors" often points to software-level architectural problems rather than hardware limitations. The most likely root causes include:

- **Connection Pool Exhaustion:** The application server or database may be configured with a small, fixed number of available connections. When the number of concurrent requests exceeds this limit, the server cannot establish new connections and immediately returns an error, even if the server's CPU and RAM are not fully utilized.
- **Concurrency Issues (Race Conditions):** There might be a bug in the application code where multiple threads attempt to access or modify a shared resource (like a session object or a cache) without proper synchronization (locking). This can lead to unexpected exceptions and server errors (HTTP 500) when the system is under load.

8.1.3 Business Impact and Implications

The consequences of this instability are severe. An error rate of 3% means that for every 100 potential user actions (like logging in or viewing a product), 3 will fail. This leads directly to:

- **Poor User Experience:** Users will be met with random, inexplicable errors, causing frustration and a loss of trust in the application.
- **Barrier to Scalability:** The application cannot be reliably scaled. Adding more users will only result in a higher absolute number of failed requests. The core software defects must be addressed before the system can handle growth.

In essence, these defects represent a critical risk to the application's viability. The next step for the development team should be to thoroughly investigate server logs corresponding to the test timestamps to identify the specific exceptions being thrown, which will provide direct clues to the root cause.

9 Use of AI Tools

9.1 AI's Role in the Project

An AI assistant (Google's Gemini) was utilized as a collaborative partner for various technical and reporting tasks. The AI served as a tool for brainstorming, code generation, and clarifying testing methodologies, which significantly accelerated the project's progress.

9.2 Key AI Interactions and Outcomes

Instead of a simple list of prompts, this section details the specific dialogues with the AI and the tangible outcomes received, which directly contributed to the project.

Interaction 1: Test Data Generation

Prompt: "I need to prepare test data for my JMeter script. Can you write an SQL script to insert 500 unique, numbered users (e.g., user1, user2) with the same password into a 'users' table?"

Outcome: The AI generated a complete, ready-to-use SQL script. This script utilized a loop or a recursive Common Table Expression (CTE) to efficiently create the 500 user records. This saved a significant amount of manual effort that would have been spent on data preparation and allowed the focus to remain on test execution.

Interaction 2: Test Strategy and Configuration

Prompt: "I'm setting up a Stress Test for the first time. What's the best way to configure the Thread Group in JMeter to effectively find my application's breaking point?"

Outcome: The AI provided a clear, step-by-step strategy. It advised against starting with an extremely high load. Instead, it recommended a gradual ramp-up (e.g., starting with 100 users and increasing to 2000 over 150 seconds). This approach was crucial as it allowed for the observation of performance degradation over time and helped pinpoint more accurately when and how the system started to fail, which was essential for the analysis in Section 5.

Interaction 3: Defining Acceptance Criteria

Prompt: "For my Load Test of 100 concurrent users, what are reasonable and standard 'Expected Results'? I need specific numbers for response time and error rate."

Outcome: The AI explained the importance of setting clear Key Performance Indicators (KPIs). It suggested industry-standard acceptance criteria for an e-commerce application, such as: "Average response time should be less than or equal to 2500ms" and "The error rate must be less than or equal to 2

9.3 Contribution and Validation

The AI's primary contributions were providing specific, actionable solutions to technical challenges and helping to define the very structure of the tests themselves.

All information and code generated by the AI were critically validated. The SQL script was executed and the data was verified in the database. The JMeter configurations were implemented and monitored to ensure they produced the intended load pattern. The acceptance criteria were reviewed to ensure they were appropriate for the project's context. This validation process ensured the AI was used as a powerful productivity tool while full human oversight and control were maintained.

10 Video Demonstration

The entire performance testing process, from script configuration in JMeter to execution and result analysis, was recorded. To provide a clear and focused view of each testing methodology, the demonstration has been split into three separate videos, one for each type of test.

The videos can be accessed via the following links:

- **Load Testing Video:** Demonstrates the execution of the 100, 200, and 500 concurrent user tests and a brief analysis of the results.
[\[Link to Load Testing Demonstration\]](#)

- **Stress Testing Video:** Shows the execution of the 700, 1000, and 2000 concurrent user tests, highlighting how the system's breaking point was identified.
[\[Link to Stress Testing Demonstration\]](#)
- **Spike Testing Video:** Displays the system's behavior during a sudden surge of 400 users and its ability to recover after the spike.
[\[Link to Spike Testing Demonstration\]](#)

11 Self-Assessment

Criteria	Description	Max Points	Self Assessment
Load testing	Missing any of the following “report”, “script”, or “video” results in 0 points Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Stress testing	Missing any of the following “report”, “script”, or “video” results in 0 points Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Spike testing	Missing any of the following “report”, “script”, or “video” results in 0 points Report: 1.0 TestCases, BugReport: 0.5 Script, Data: 0.5 Video: 1.0	3.0	3.0
Use of AI Tools	Prompt transparency, critical validation, added value	1.0	1.0
Total		10.0	10.0